

Manuel utilisateur de autolisp-hash-table

OpenAI

Objet

`autolisp-hash-table` fournit une API de tables de hachage inspirée de Common Lisp, adaptée à AutoLISP.

La v1 propose :

- création de tables ;
- insertion et lecture par clé ;
- suppression ;
- vidage ;
- itération ;
- fonction de hachage `ah-sxhash`.

Chargement

Le module dépend actuellement de `autolisp-vector` pour son stockage interne.

Exemple de chargement :

```
(load "../autolisp-vector/src/autolisp-vector.lsp")  
(load "src/autolisp-hash-table.lsp")
```

API publique

Création

```
(ah-make-hash-table [test [size [rehash-size [rehash-threshold]]]])
```

Crée une table de hachage.

Arguments :

- test** symbole, actuellement `eq` ou `equal` ;
- size** capacité cible initiale ;
- rehash-size** multiplicateur de croissance ;
- rehash-threshold** seuil de charge.

Exemple :

```
(setq table (ah-make-hash-table 'equal 23 2.0 0.5))
```

Prédicats et introspection

- (ah-hash-table-p table)
- (ah-hash-table-count table)
- (ah-hash-table-size table)
- (ah-hash-table-test table)
- (ah-hash-table-rehash-threshold table)

Accès

(ah-gethash table key [default])

Retourne la **valeur principale**.

Effet secondaire :

- met à jour la variable globale **ah-*gethash-found-p*** ;
- cette variable vaut T si la clé existe ;
- elle vaut nil sinon.

Exemples :

```
(ah-gethash table "foo")  
; => nil  
ah-*gethash-found-p*  
; => nil
```

```
(ah-gethash table "foo" 42)  
; => 42  
ah-*gethash-found-p*  
; => nil
```

(ah-remhash table key)

Retourne la **valeur supprimée**.

Effet secondaire :

- met à jour la variable globale **ah-*remhash-found-p*** ;
- cette variable vaut T si la clé existait ;
- elle vaut nil sinon.

Exemples :

```
(ah-remhash table "foo")  
; => 10  
ah-*remhash-found-p*
```

```
; => T
```

```
(ah-remhash table "unknown")  
; => nil  
ah-*remhash-found-p*  
; => nil
```

Mutation

- (ah-puthash table key value)
- (ah-remhash table key)
- (ah-clrhash table)

Exemples :

```
(ah-puthash table "foo" 10)  
(ah-gethash table "foo")  
; => 10  
ah-*gethash-found-p*  
; => T
```

```
(ah-remhash table "foo")  
; => 10  
ah-*remhash-found-p*  
; => T
```

Itération

```
(ah-maphash function table)
```

Appelle function sur chaque couple clé/valeur.

Exemple :

```
(ah-maphash  
  (function  
    (lambda (k v)  
      (print (list k v))))  
  table)
```

Hachage

```
(ah-sxhash object [test])
```

Retourne un entier non négatif stable dans la session courante.

Exemples :

```
(ah-sxhash "abc")  
(ah-sxhash '(1 2 3) 'equal)
```

Types pris en charge

La v1 prend en charge :

- `nil` ;
- nombres ;
- chaînes ;
- symboles ;
- listes et dotted pairs ;
- objets CAD via représentation observable stable.

Règle générale :

- le hash combine toujours un **tag de type** ;
- puis une **représentation observable stable**.

Exemples :

symbole nom du symbole ;
entité CAD type + handle ;
objet inconnu représentation imprimée documentée.

Sémantique des tests

Les tests supportés sont :

- `eq`
- `equal`

Le hash utilisé dépend du test de la table.

Limitations de la v1

- pas d'API compatible `setf` ;
- pas de garantie particulière sur l'ordre d'itération ;
- pas de gestion documentée des structures circulaires dans `ah-sxhash` ;
- stockage interne encore fondé sur `autolisp-vector`.

Benchmarks

Le dépôt contient un banc d'essai dans `tests/hash-table-benchmarks.lsp`.

État observé au 14 mars 2026 :

- `make test-bricscad` passe ;
- `make bench-bricscad` passe en utilisant le runner dédié `tests/run-benchmarks.lsp` ;

- sur macOS + BricsCAD, il reste préférable d'utiliser l'espace de travail **2D Drafting** et non **2D Drafting (Modern)**.

Mesures relevées le 14 mars 2026 :

opération	taille	itérations	ms	note
ah-puthash	11	11	2	count=11
ah-gethash	11	11	1	sum=55
ah-remhash	11	11	1	count=0
ah-puthash	23	23	4	count=23
ah-gethash	23	23	1	sum=253
ah-remhash	23	23	2	count=0
ah-puthash	47	47	10	count=47
ah-gethash	47	47	5	sum=1081
ah-remhash	47	47	6	count=0
ah-puthash	97	97	65	count=97
ah-gethash	97	97	46	sum=4656
ah-remhash	97	97	47	count=0

Ces mesures sont des mesures de développement locales. Elles sont utiles pour suivre les régressions et comparer les opérations entre elles, mais elles ne constituent pas encore un engagement de performance normatif.

Exemple complet

```
(setq table (ah-make-hash-table 'equal 11 2.0 0.5))

(ah-puthash table "a" 1)
(ah-puthash table "b" 2)

(ah-gethash table "a")
; => 1
ah-*gethash-found-p*
; => T

(ah-gethash table "z" 'missing)
; => missing
ah-*gethash-found-p*
; => nil

(ah-remhash table "b")

(ah-hash-table-count table)
; => 1
```